

Reporte de
PRÁCTICA 01



SECCION CRITICA

**CONCURRENCIA Y
PARALELISMO**

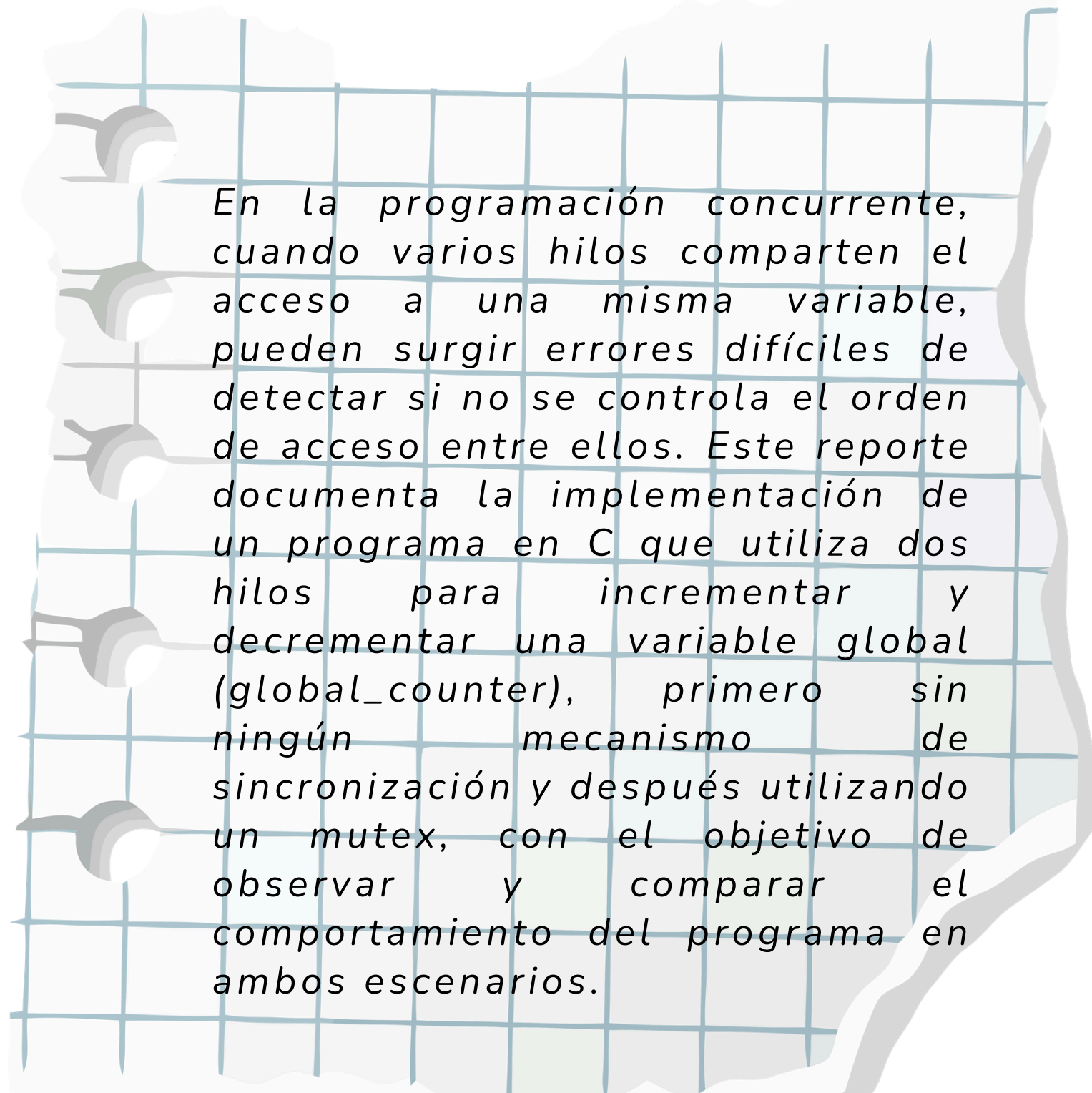
**INGENIERIA EN TECNOLOGIA
DE SOFTWARE**

ALAN CAMPOS SUAREZ

UNIVERSIDAD AUTONOMA DE CAMPECHE



INTRODUCCION



En la programación concurrente, cuando varios hilos comparten el acceso a una misma variable, pueden surgir errores difíciles de detectar si no se controla el orden de acceso entre ellos. Este reporte documenta la implementación de un programa en C que utiliza dos hilos para incrementar y decrementar una variable global (`global_counter`), primero sin ningún mecanismo de sincronización y después utilizando un mutex, con el objetivo de observar y comparar el comportamiento del programa en ambos escenarios.



OBJETIVO

Implementar el pseudocódigo proporcionado para experimentar con diferentes mecanismos de sincronización en programación concurrente, con el fin de comprender los conceptos de condición de carrera y sección crítica en hilos.



PSEUDOCODIGO

```
int global_counter=20;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *thread_routine(void *arg){
    for (size_t i = 0; i < 1000; i++)
    {
        global_counter ++;
    }
}

void *thread_routine_two(void *arg){
    for (size_t i = 0; i < 1000; i++)
    {
        global_counter --;
    }
}
```

Se tomó el pseudocódigo proporcionado y se implementó en un archivo **programa1.c**. El programa declara `global_counter` inicializado en 20, y crea dos hilos: uno ejecuta un ciclo de 1000 iteraciones donde incrementa la variable (`global_counter++`), y el otro ejecuta un ciclo de 1000 iteraciones donde la decrementa (`global_counter--`). Al finalizar ambos hilos, el programa imprime el valor final de `global_counter`.



CONDICION DE CARRERA

Ocurre cuando dos o más hilos acceden y modifican una misma variable compartida al mismo tiempo, sin control de acceso entre ellos. Como `global_counter++` se traduce en tres instrucciones no atómicas (`movl`, `addl/subl`, `movl`), un hilo puede ser interrumpido a mitad de esos pasos, provocando que se pierdan actualizaciones y que el resultado final sea impredecible entre ejecuciones.

```
alancs@AlanCampos:~/practica01$ gcc -o race_test race_test.c -lpthread
alancs@AlanCampos:~/practica01$ ./race_test
Valor final de global_counter: 20
alancs@AlanCampos:~/practica01$ ./race_test
Valor final de global_counter: 20
alancs@AlanCampos:~/practica01$ ./race_test
Valor final de global_counter: -980
```

Se compiló y ejecutó el programa varias veces consecutivas sin ningún mecanismo de sincronización. Aunque en teoría el resultado siempre debería ser 20 (ya que un hilo suma 1000 y el otro resta 1000), en la práctica el resultado varió entre ejecuciones: la mayoría dio 20, pero una ejecución arrojó -980. Esto evidencia la condición de carrera.



SECCION CRITICA

La sección crítica es la parte del código donde los hilos acceden y modifican un recurso compartido. En este programa, la sección crítica es el bloque ``for`` que modifica ``global_counter`` (``global_counter++`` en un hilo, ``global_counter--`` en el otro).

Al agregar el mutex, la sección crítica queda claramente delimitada: es el bloque `for` que modifica `global_counter` (`global_counter++` en un hilo, `global_counter--` en el otro), ahora protegido entre `pthread_mutex_lock()` y `pthread_mutex_unlock()`. Esto garantiza que solo un hilo pueda ejecutar ese bloque a la vez, evitando que ambos modifiquen la variable simultáneamente y eliminando la condición de carrera.

```
pthread_mutex_unlock(&mutex);
```



```
alancs@AlanCampos:~/practica01$ ./programa1
Starting thread...
Starting second thread...
Valor final de global_counter: 20
```

ANALISIS ENSAMBLADOR

Para entender por qué ocurre la condición de carrera, se generó el código ensamblador con: `gcc -S race_test.c`

```
alancs@AlanCampos:~/practica01$ gcc -S race_test.c
```

Al analizar el archivo `race_test.s`, se observó que `global_counter++` se traduce en tres instrucciones:

```
.L3:
    movl    global_counter(%rip), %eax
    addl    $1, %eax
    movl    %eax, global_counter(%rip)
    addq    $1, -8(%rbp)
```

movl: Copia el valor de `global_counter` de la memoria a un registro del CPU

addl: Suma 1 al registro (incremento)

movl: Guarda el valor modificado de vuelta en la memoria



Esto confirma que `global_counter++` no es una operación atómica, sino que requiere 3 pasos. El sistema operativo puede interrumpir un hilo entre estos pasos, permitiendo que otro hilo modifique la variable y cause pérdida de datos. El mutex protege estos 3 pasos como un solo bloque.

EVIDENCIA DE FUNCIONAMIENTO

Además del valor correcto de `'global_counter'` (20), el archivo `'readme.txt'` muestra que la sección crítica está protegida:

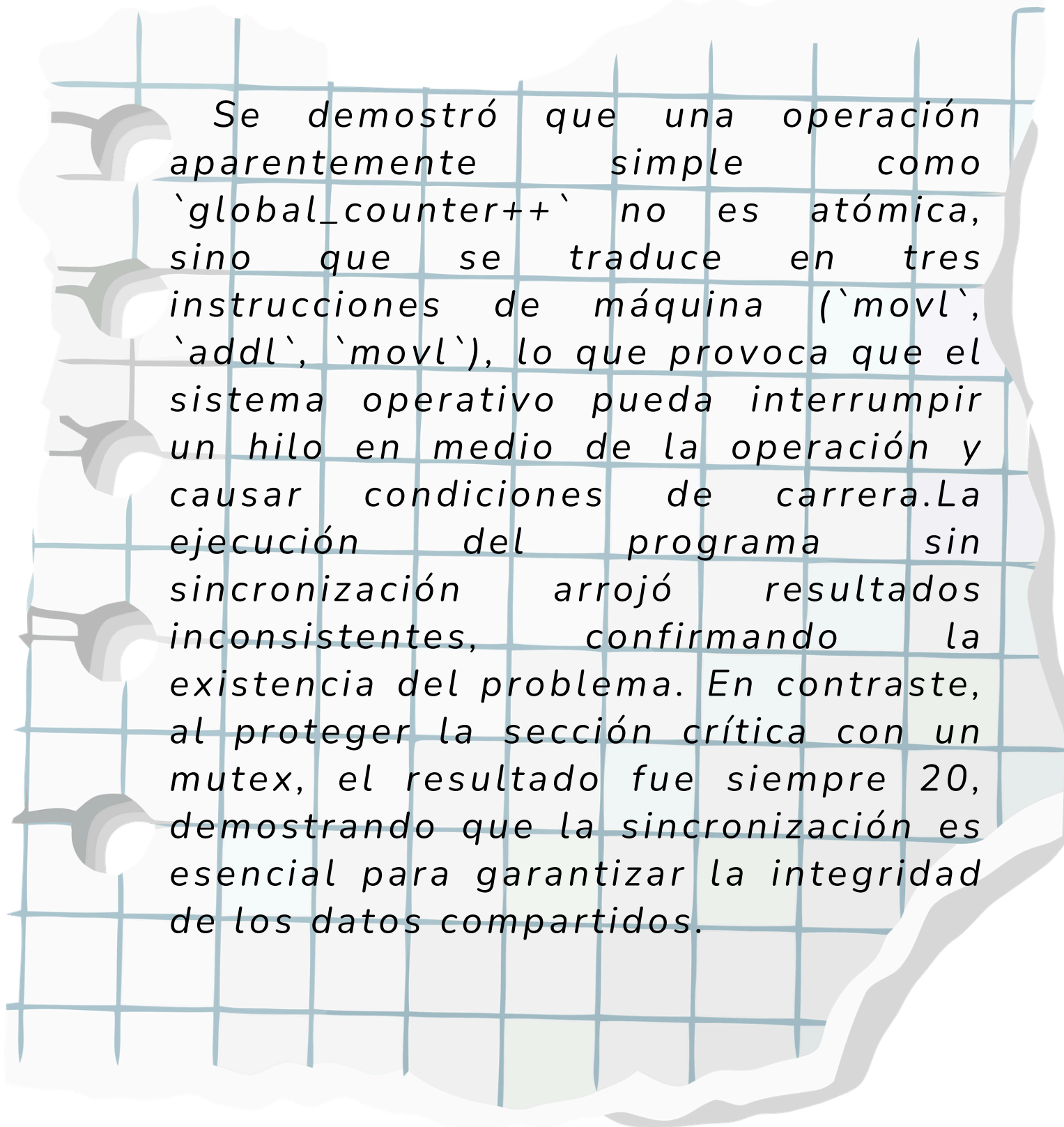
```
alan@AlanCampos:~/practica01$ cat ~/CyP_2026/readme.txt
```



Hilo 1 - Linea 986
Hilo 1 - Linea 987
Hilo 1 - Linea 988
Hilo 1 - Linea 989
Hilo 1 - Linea 990
Hilo 1 - Linea 991
Hilo 1 - Linea 992
Hilo 1 - Linea 993
Hilo 1 - Linea 994
Hilo 1 - Linea 995
Hilo 1 - Linea 996
Hilo 1 - Linea 997
Hilo 1 - Linea 998
Hilo 1 - Linea 999
Hilo 1 - Linea 1000
Hilo 2 - Linea 1001
Hilo 2 - Linea 1002
Hilo 2 - Linea 1003
Hilo 2 - Linea 1004
Hilo 2 - Linea 1005
Hilo 2 - Linea 1006
Hilo 2 - Linea 1007
Hilo 2 - Linea 1008
Hilo 2 - Linea 1009
Hilo 2 - Linea 1010
Hilo 2 - Linea 1011
Hilo 2 - Linea 1012
Hilo 2 - Linea 1013
Hilo 2 - Linea 1014



CONCLUSION



Se demostró que una operación aparentemente simple como ``global_counter++`` no es atómica, sino que se traduce en tres instrucciones de máquina (``movl``, ``addl``, ``movl``), lo que provoca que el sistema operativo pueda interrumpir un hilo en medio de la operación y causar condiciones de carrera. La ejecución del programa sin sincronización arrojó resultados inconsistentes, confirmando la existencia del problema. En contraste, al proteger la sección crítica con un mutex, el resultado fue siempre 20, demostrando que la sincronización es esencial para garantizar la integridad de los datos compartidos.